

Algoritmi e Strutture Dati – Parte B – 25/08/25

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Mensa – Punti ≥ 8

Dopo il grande successo nella gestione delle macchinette del caffè, dal prossimo anno Illiria prenderà in gestione anche il servizio mensa. D'ora in poi, gli studenti che intendono mangiare in mensa dovranno selezionare tramite un'app i piatti che sono disposti a mangiare. Siete stati assunti da Illiria e il vostro compito per ottimizzare il grado di soddisfazione degli studenti.

Le informazioni a disposizione sono:

- l'insieme S degli studenti;
- l'insieme P dei piatti disponibili; ciascun piatto $p \in P$ è caratterizzato da
 - il suo *tipo* $t_p \in \{\text{primo, secondo, dessert}\}$;
 - il numero di porzioni disponibili $m_p \geq 1$;
- per ogni studente $s \in S$, l'elenco $A_s \subseteq P$ dei piatti che accetterebbe di mangiare.

A ogni studente possono essere assegnati al più un primo, al più un secondo e al più un dessert tra quelli che accetta di mangiare. Se non è possibile soddisfare tutte le richieste e uno studente riceve un pasto a cui manca qualche portata, pagherà solo ciò che consuma.

Descrivere un algoritmo che calcoli il *massimo numero di assegnazioni studente-piatto* che rispettino i vincoli di cui sopra.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

B2 – Sottoinsiemi k -distanti – Punti ≥ 10

Scrivere un algoritmo

```
printDistantSets(int[] A, int n, int k)
```

che prenda in input un vettore A contenente n interi (non necessariamente distinti) e un intero positivo k , e che stampi tutti i sottoinsiemi k -distanti di A , evitando duplicati.

Un sottoinsieme $S \subseteq A$ si dice k -*distante* se presi due elementi v_1 e v_2 da S , questi distano più di k , ovvero:

$$\forall v_1, v_2 \in S, v_1 \neq v_2 \implies |v_1 - v_2| > k.$$

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Esempio. Se $A = [2, 4, -2, 5, 2]$ e $k = 2$, i sottoinsiemi 2-distanti di A sono: $\emptyset$, $\{2\}$, $\{4\}$, $\{-2\}$, $\{5\}$, $\{-2, 2\}$, $\{-2, 4\}$, $\{-2, 5\}$, $\{2, 5\}$, $\{-2, 2, 5\}$.

B3 – Going nowhere – Punti ≥ 12

Si consideri una griglia lineare composta da un numero infinito di celle, numerate da $-\infty$ a $+\infty$. Una pedina si trova inizialmente nella cella 0 e può effettuare tre tipi di mosse: andare a sinistra (dalla cella i alla cella $i - 1$, mossa L), a destra (dalla cella i alla cella $i + 1$, mossa R), oppure restare ferma (dalla cella i alla cella i , mossa S per *stay*).

Scrivere un algoritmo

```
int countWays(int n)
```

che prenda in input un numero intero n e restituisca il numero di sequenze di n mosse che, partendo dalla cella 0, riportano la pedina nella cella 0.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, se $n = 3$, le possibili sequenze di mosse che riportano la pedina nella cella 0 partendo dalla cella 0 sono: LRS, LSR, RLS, RSL, SLR, SRL, SSS, per un totale di 7 sequenze. L'algoritmo deve quindi restituire 7.

I valori da restituire per n che va da 1 a 10 sono 1, 3, 7, 19, 51, 141, 393, 1107, 3139, 8953.